

Afterburn V 0.1: Definitive Codebase Encyclopedia

Current Version: 4.0 (Ultimate Deep Dive)

Scope: Full Function-Level Documentation

This document is a complete technical breakdown of the engine. It describes **every function** in **every file**, explaining execution flow, dependencies, and internal logic without summarization.

1. EXECUTION FLOW & CORE LOOP (`iMain.cpp`)

The application entry point and central orchestrator.

Function: `main()`

- **Purpose:** The entry point of the C++ program. Initializes the window and starts the event loop.
- **Logic Trace:**

1. `iInitialize(1920, 1080, "Afterburn V 0.1")` : Creates the OpenGL window.
2. `iSetTimer(16, gameUpdate)` : Registers `gameUpdate` as a callback to run every 16ms (~60 FPS).
3. `menuInit()` : Loads menu assets (textures).
4. `gameInit()` : Calls init functions for sub-systems (Road, UI, Player, Enemy).
5. `iStart()` : Enters the infinite GLUT processing loop. **The function never returns.**

Function: `gameUpdate()`

- **Trigger:** Timer Callback (every 16ms).

- **Purpose:** The logical "Brain" of the game. Updates positions and states.
- **Logic Trace:**

1. **State Check:** `if (gameState == STATE_GAME)`

2. **Sub-System Updates:**

- * Calls `playerUpdate()` : Moves the car.
- * Calls `roadUpdate()` : Scrolls the background.
- * Calls `enemyUpdate()` : Runs AI and Collision checks.

3. **Game Over Logic:**

- * Checks: `player.health <= 0` AND Boss is NOT dying (`!isBossDying()`).
- * Action: Sets `gameState = STATE_GAMEOVER`.
- **Dependencies:** Requires `Player.h` (for health), `Enemy.h` (for updates).

Function: `iDraw()`

- **Trigger:** Continuous Rendering Loop.
- **Purpose:** The "Painter". Clears the screen and draws the current state.
- **Logic Trace:**

1. `iClear()` : Wipes the buffer black.

2. `switch (gameState)` :

- * `STATE_MENU` : Calls `menuDraw()`.
- * `STATE_GAME` / `STATE_GAMEOVER` : Calls `gameDraw()`.
- * `STATE_OPTIONS` : Calls `optionsDraw()`.

- **Dependencies:** Requires `menu.h`, `UI.h`, `Road.h`, `Player.h`, `Enemy.h`.

Function: `gameDraw()`

- **Trigger:** Called by `iDraw` when state is GAME.
- **Purpose:** Renders the active gameplay layer.
- **Logic Trace:**

1. `roadDraw()` : Draws background (Layer 0).
2. `playerDraw()` : Draws car/bullets (Layer 1).
3. `enemyDraw()` : Draws enemies/effects (Layer 1).
4. `uiDraw()` : Draws HUD (Layer 2 - Top).

2. PLAYER MODULE (Player.h)

Handles the user inputs and physics simulation for the player entity.

Struct: `PlayerState`

- `float x, y` : World coordinates.
- `float angle` : Visual rotation (Degrees).
- `float shootAngle` : Aiming vector angle (Radians).
- `bool isFiring` : Trigger state.
- `int shootTimer` : Frames until next shot.

Function: `playerUpdate()`

- **Trigger:** Called by `gameUpdate` every frame.
- **Step-by-Step Logic:**

1. **Active Check:** `if (!player.active) return;`

2. **Movement Handling:**

* Reads `w, a, s, d` keys via `isKeyPressed()`.

* Increments `player.x` or `player.y` by `PLAYER_SPEED`.

* **Boundary Enforcement:**

* `if (x < ROAD_LEFT_LIMIT) -> x = ROAD_LEFT_LIMIT;`

* Prevents driving off the texture.

3. Rotation Calculation:

* Calculates `dx = mouseX - CarCenter`, `dy = mouseY - CarCenter`.

* `shootAngle = atan2(dy, dx)`.

`angle = shootAngle * 180 / PI`.

* `player.spriteIndex = getFrameFromAngle(...)`.

4. Shooting Logic:

* `shootTimer--`.

* If `isFiring` AND `shootTimer <= 0`: Calls `playerShoot()`.

5. Projectile Updates:

* Loops `std::vector<Projectile>`.

* `p.x += p.dx`, `p.y += p.dy`.

* If off-screen limits are breached, sets `p.active = false`.

Function: `playerShoot()`

- **Trigger:** Inside `playerUpdate` when allowed.
- **Logic:**

1. Creates `Projectile p`.
2. Sets `p.x`, `p.y` to car center.
3. Calculates Normalized Vector: `(dx/Len, dy/Len)`.
4. Sets `p.dx`, `p.dy` to `Vector * Speed (10.0f)`.
5. Pushes to `projectiles` vector.

3. ENEMY & AI MODULE (`Enemy.h`)

Manages spawning, AI behavior, collision resolution, and Boss logic.

Function: `enemySpawn()`

- **Trigger:** `enemyUpdate` (Every frame).
- **Logic:**
 1. **Population Cap:** Uses `score` to determine `maxEnemies` (3 or 5).
 2. **Spawn Condition:** If `activeCount < maxEnemies`.
 3. **Instantiate:**
 - * `Enemy e;`
 - * `e.x`: Random range inside road limits.
 - * `e.y`: `-(ENEMY_HEIGHT + Random(400))`. (Spawns BELOW screen).
 4. **Special Enemy Logic:**
 - * Checks `score`. If `score >= 500`:
 - * Spawns 2 `TYPE_SPECIAL` enemies.
 - * These have distinct AI logic in `enemyUpdate`.

Function: `enemyUpdate()`

- **Trigger:** `gameUpdate` (Every frame).
- **Purpose:** The central simulation step for non-player entities.
- **Step-by-Step Logic:**
 1. **Explosion Update:** Iterates `explosions` vector, increments `frameIndex`. Removes if animation finishes.
 2. **Phase Manager:**
 - * If `score >= 1000` -> `PHASE_NORMAL`.
 - * If `PHASE_WARNING` -> Accelerates all enemies away (`y += 15`). Clears `puddles`. When empty -> `PHASE_TRUCK`.

* If `PHASE_TRUCK` -> Updates `SupplyTruck` logic (Enter -> Drop Orb -> Leave).

* If `PHASE_BOSS` -> Runs Boss State Machine.

3. Collision Checks (Main Loop):

* **Projectile vs Enemy:**

* Checks intersection `checkAABB`.

* If hit: `Enemy.health -= 5`.

* If dead: Spawn Explosion, Add Score (`+10`).

* **Player vs Enemy:**

* Checks intersection `checkAABB`.

* If hit: `Player.health -= 5`, `Enemy.health -= 2`.

* **Bounce:** `Enemy.y -= 20`.

Function: `checkAABB(...)`

- **Purpose:** Axis-Aligned Bounding Box collision detection.
- **Parameters:** `x1, y1, w1, h1` (Box A), `x2, y2, w2, h2` (Box B).
- **Logic:**

cpp

```
return (x1 < x2 + w2 && x1 + w1 > x2 && y1 < y2 + h2 && y1 + h1 > y2);
```

* Returns true **only if** rectangles overlap on both axes.

🔗 4. ROAD MODULE (Road.h)

Manages the scrolling background illusion.

Function: roadUpdate()

- **Trigger:** gameUpdate .
- **Logic:**

1. `roadY -= ROAD_SPEED` (Moves texture coordinate down).

2. Loop Logic:

* `if (roadY <= -1080)` : Checks if the image has fully scrolled past.

* `roadY = 0` : Snaps back to top.

* `player.score += 10` : Increments score for "distance traveled".

Function: roadDraw()

- **Trigger:** gameDraw .
- **Logic:**

1. Draw Image 1 at `(0, roadY)` .

2. Draw Image 2 at `(0, roadY + 1080)` .

* **Reasoning:** We need two images to cover the screen wrapping point seamlessly.

🔗 5. UI MODULE (UI.h)

Handles the Heads-Up Display (HUD) and Overlay Screens.

Function: uiDraw()

- **Trigger:** iDraw (Called last, on top of everything).
- **Logic Trace:**

1. Health Bar:

* Calculates `hpRatio = player.health / 100.0`.

Draws Red Rectangle: Width = `315 * hpRatio`.

* Draws Texture Frame on top.

2. Score:

* Formats string "Score: %d".

* Calls `iText` to render string.

3. Screens:

* If `PHASE_WIN`: Draws Background + Text "Congratulations".

* If `STATE_GAMEOVER`: Draws "YOU LOSE" text.

6. MENU MODULE (`menu.cpp`)

Handles interactive buttons and scene switching.

Function: `menuInit()`

- **Trigger:** `main`.

- **Logic:**

* Loads button textures ("Asset/Menu files/...").

* Initializes `buttons[]` array with `x`, `y`, `w`, `h` and `targetState` (e.g., `STATE_GAME`, `STATE_OPTIONS`).

Function: `menuDraw()`

- **Trigger:** `iDraw` (when `STATE_MENU`).

- **Logic:**

* Calculates `dt` (Delta Time) for smooth animation.

- * Iterates `buttons[]`.
- * **Interpolation:** Moves `currentScale` towards `targetScale` (1.0 or 1.08).
- * Draws Scaled Image centered at button position.

Function: `menuMouseClicked(mx, my)`

- **Trigger:** `iMouse` (Left Click).
 - **Logic:**
- * Iterates `buttons[]`.
 - * Checks collision `pointInButton`.
 - * If valid:
 - * Updates global `currentState`.
 - * Returns new state to `iMain.cpp`.
 - * Special Case: If `target == STATE_EXIT`, calls `exit(0)`.

7. OPTIONS MODULE (`options.cpp`)

Handles settings and the Controls screen.

Function: `optionsDraw()`

- **Trigger:** `iDraw` (when `STATE_OPTIONS`).
 - **Logic:**
- * Checks `currentScreen` (Main vs Controls).
 - * If Main: Draws Music Toggle, Controls Button, Back Button.
 - * If Controls: Draws static Control layout image and Back Button.

Function: `optionsMouseClicked()`

- **Trigger:** `iMouse` .
- **Logic:**

* Checks collision with `btnMusic` .

* Toggles `isMusicOn = !isMusicOn` .

* Checks collision with `btnControls` .

* Sets `currentScreen = SCREEN_CONTROLS` .

* Checks collision with `btnBack` .

* Sets `gameState = STATE_MENU` .

ðŸ”– 8. CAUSE & EFFECT RELATIONSHIPS

How changing one variable ripples through the system.

Scenario A: Changing `PLAYER_SPEED`

- **Action:** Change `#define PLAYER_SPEED` in `Player.h` from 5.0 to 10.0.
- **Immediate Effect:** `playerUpdate` adds larger increments to `player.x/y` .
- **Side Effect:**

1. Player reaches screen edges faster.
2. Player can dodge Projectiles and Enemies easier.
3. Does **not** affect shooting speed or enemy speed.

Scenario B: Increasing `ROAD_SPEED`

- **Action:** Change `#define ROAD_SPEED` in `Road.h` .
- **Immediate Effect:** The background scrolls faster in `roadDraw` .
- **Side Effect:**

1. The game *looks* faster.

2. **Gameplay Mismatch:** Enemies still move at `ENEMY_SPEED_BASE` . They will appear to be "sliding" on the road rather than driving, breaking immersion. You MUST increase `ENEMY_SPEED_BASE` to match `ROAD_SPEED` updates.

Scenario C: Changing `boss.maxHealth`

- **Action:** Change value in `enemyReset()` .
- **Immediate Effect:** Boss takes more hits to kill.
- **Side Effect (UI):**

* The UI calculation `boss.health / 2` assumes MaxHP is 200.

* If you change MaxHP to 500, the UI will show "250%" health initially.

* **Fix:** You must update the UI string formatting logic in `UI.h` .
